Eye of Sauron Smart Auto-Patcher

A production-grade JavaScript/TypeScript AST analyzer and auto-patcher that safely fixes low-risk issues while reporting higher-risk problems for manual review.

Features

✅ Safe Automatic Patching

- Removes unguarded console statements
- Prunes unused imports
- Replaces deprecated Webpack plugins

⚠️ Smart Detection & Reporting

- Detects hardcoded secrets using entropy analysis
- Identifies unguarded await expressions
- Finds unpaired event listeners
- Supports custom detectors

🛡️ Safety First

- Creates backups before any modifications
- Only auto-fixes LOW risk issues with high confidence
- Comprehensive JSON reports
- Respects configuration rules

Installation

```bash
# Install dependencies
npm install

# Or with specific Babel packages
npm install @babel/parser @babel/traverse @babel/generator @babel/types
```

Usage

Basic Usage

```bash
# Scan default source directory (./src)
node smartAutoPatch.js

# Scan specific directory
node smartAutoPatch.js --path ./lib

# Use custom configuration
node smartAutoPatch.js --path ./src --config custom.config.json
```

NPM Scripts

```bash
# If you've set up package.json scripts
npm run patch
npm run patch:src
npm run patch:lib
```

Configuration

The patcher uses `eos.config.json` for configuration. Key options:

```json
{
  "allowConsole": false,          // Allow console statements globally
  "webpackVersion": "5.85.0",     // Target Webpack version
  "ignorePatterns": [             // Strings to ignore in secret detection
    "PLACEHOLDER",
    "YOUR_API_KEY_HERE"
  ],
  "exclude": [                    // Directories/files to skip
    "node_modules",
    "dist",
    "**/*.min.js"
  ],
  "include": [                    // Patterns to include
    "src/**/*.js",
    "lib/**/*.js"
  ],
  "rules": {                      // Detection rules
    "console": {
      "enabled": true,
      "autoFix": true             // Auto-remove unguarded console
    },
    "hardcodedSecrets": {
      "enabled": true,
      "autoFix": false            // Never auto-fix secrets
    }
  }
}
```

Risk Levels

The patcher categorizes issues into three risk levels:

- **LOW** (Green): Safe to auto-fix
  - Unguarded console statements
  - Unused imports
  - Deprecated Webpack plugins
- **MEDIUM** (Yellow): Requires review
  - Unguarded await expressions
  - Unpaired event listeners

- **HIGH** (Red): Immediate attention needed
  - Hardcoded secrets and API keys

Output

Console Output

```
🔍 Eye of Sauron Smart Auto-Patcher v1.0.0
═══════════════════════════════════════════

📁 Target: /path/to/project/src
⚙️  Config: patch-report.json

📋 Found 25 files to scan

📄 src/utils/api.js: 3 issues
📄 src/components/Logger.js: 2 issues

📋 PATCH PLAN
═══════════════════════════════════════════

✅ Auto-fix (5 issues):
  📄 src/utils/api.js
    Line 15: Unguarded console.log statement found
    Line 23: Unused import: lodash from 'lodash'

⚠️  Manual review needed (2 issues):
  📄 src/services/auth.js
    Line 45: Await expression not wrapped in try-catch block
      💡 Wrap await expression in try-catch block for proper error handling

🚫 High-risk issues requiring immediate attention (1 issues):
  📄 src/config/keys.js
    Line 3: Potential hardcoded secret detected in 'apiKey'
      💡 Move to environment variable: process.env.API_KEY
```

JSON Report (patch-report.json)

```json
{
  "version": "1.0.0",
  "timestamp": "2025-01-01T12:00:00.000Z",
  "duration": "1523ms",
  "summary": {
    "filesScanned": 25,
    "issuesFound": 8,
    "fixesApplied": 5,
    "issuesSkipped": 3
  },
  "patches": [...],
  "pendingReview": [...],
  "highRiskIssues": [...]
}
```

Custom Detectors

Create custom detectors by implementing the detector interface:

```javascript
// src/utils/myDetector.js
export default {
  name: 'myDetector',
  async detect(ast, content, filePath, config) {
    const issues = [];
    // Your detection logic here
    return issues;
  }
};
```

Then add to your config:

```json
{
  "customDetectors": [
    "src/utils/myDetector.js"
  ]
}
```

Exit Codes

- `0`: Success - all issues fixed or no issues found
- `1`: Issues need manual review
- `2`: High-risk issues detected
- `3`: Fatal error

Backup and Recovery

All modified files are backed up to `.eos-backups/` with timestamps:

```
.eos-backups/
├── src/
│   ├── api.js.2025-01-01T12-00-00-000Z.backup
│   └── logger.js.2025-01-01T12-00-00-000Z.backup
```

Best Practices

1. **Always review the patch plan** before letting it apply fixes
2. **Check the JSON report** for detailed information about all issues
3. **Address high-risk issues immediately** - these are never auto-fixed
4. **Test your code** after patches are applied
5. **Keep backups** until you're satisfied with the changes

Troubleshooting

Parse Errors

If you see parse errors, ensure your code is valid JavaScript/TypeScript. The patcher uses error recovery but extremely malformed code may fail.

Large Files

Files over 1MB are skipped by default. Adjust `maxFileSize` in config if needed.

Custom Detectors Not Loading

Ensure the path is correct and the detector exports the correct interface.

License

MIT